

Kubernetes Setup - Hands-On Lab (HOL)

Trainer: Dr. Sandeep Kumar Sharma

Learning Objectives

By the end of this Hands-On Lab, participants will be able to:

- Understand how to provision infrastructure for Kubernetes cluster setup
 - Install and configure Kubernetes components using kubeadm
 - Set up a control plane (master node) and worker nodes
 - Configure container runtime using Docker and cri-dockerd
 - Initialize a Kubernetes cluster
 - Deploy a networking solution (Calico)
 - Join worker nodes to the Kubernetes cluster
 - Verify cluster health and node status
-

Expected Output

At the end of this lab:

- A fully working Kubernetes cluster (1 Master + 2 Workers)
 - Nodes should be in **Ready** state
 - Pods should be successfully deployed across nodes
 - Networking (Calico) should be functional
-

Environment Details

- **OS:** Ubuntu 22.04
 - **Cluster Setup:** 1 Master + 2 Worker Nodes
 - **Platform:** AWS EC2 (Same VPC/Network)
-

Step 1: Create EC2 Instances

Create **3 Ubuntu 22.04 machines** in AWS:

- 1 Master Node
- 2 Worker Nodes

⚠ Ensure all machines:

- Are in the same VPC
 - Can communicate via private IP
 - Have required ports open (6443, 10250, etc.)
-

Step 2: Change Hostnames

Run below command on each machine:

```
hostnamectl set-hostname master
```

```
hostnamectl set-hostname worker1
```

```
hostnamectl set-hostname worker2
```

Step 3: Bootstrap Script

Create a script on each machine:

```
vi script.sh
```

Paste the following:

```
#!/bin/bash
K8S_MINOR="v1.33"
K8S_PKG_VERSION="1.33.5-1.1"
CRID_VER="0.3.19"

sudo apt update -y
sudo apt install -y apt-transport-https ca-certificates curl gnupg lsb-release
software-properties-common wget

sudo apt install -y docker.io
sudo systemctl enable --now docker

sudo modprobe overlay || true
sudo modprobe br_netfilter || true
```

```

cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward = 1
EOF

sudo sysctl --system

sudo mkdir -p -m 755 /etc/apt/keyrings
curl -fsSL "https://pkgs.k8s.io/core:/stable:${K8S_MINOR}/deb/Release.key" |
sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://
pkgs.k8s.io/core:/stable:${K8S_MINOR}/deb/ /" | sudo tee /etc/apt/
sources.list.d/kubernetes.list

sudo apt-get update
sudo apt-get install -y kubelet=${K8S_PKG_VERSION} kubeadm=${K8S_PKG_VERSION}
kubectl=${K8S_PKG_VERSION}

TMPDIR=$(mktemp -d)
cd "${TMPDIR}"

CRID_TGZ="cri-dockerd-${CRID_VER}.amd64.tgz"
CRID_URL="https://github.com/Mirantis/cri-dockerd/releases/download/v$
${CRID_VER}/${CRID_TGZ}"

wget -q --show-progress "${CRID_URL}"
tar -xvf "${CRID_TGZ}"

sudo install -o root -g root -m 0755 cri-dockerd/cri-dockerd /usr/local/bin/cri-
dockerd

sudo wget -q -O /etc/systemd/system/cri-docker.service
"https://raw.githubusercontent.com/Mirantis/cri-dockerd/v${CRID_VER}/packaging/
systemd/cri-docker.service"

sudo wget -q -O /etc/systemd/system/cri-docker.socket
"https://raw.githubusercontent.com/Mirantis/cri-dockerd/v${CRID_VER}/packaging/
systemd/cri-docker.socket"

sudo sed -i -e 's|/usr/bin/cri-dockerd|/usr/local/bin/cri-dockerd|g' /etc/
systemd/system/cri-docker.service

sudo systemctl daemon-reload
sudo systemctl enable --now cri-docker.socket
sudo systemctl start cri-docker.service

```

```
kubeadm version -o short  
docker --version
```

Run the script:

```
sh script.sh
```

Step 4: Create Config File (RUN only on Master Machine)

```
vi config.yaml
```

```
apiVersion: kubeadm.k8s.io/v1beta3  
kind: InitConfiguration  
localAPIEndpoint:  
  advertiseAddress: <MASTER_PRIVATE_IP>  
  bindPort: 6443  
nodeRegistration:  
  criSocket: unix:///var/run/cri-dockerd.sock  
---  
apiVersion: kubeadm.k8s.io/v1beta3  
kind: ClusterConfiguration  
kubernetesVersion: v1.33.5  
networking:  
  podSubnet: 10.10.0.0/16
```

Step 5: Initialize Cluster (RUN only on Master Machine)

```
sudo kubeadm init --config=config.yaml >> cluster_initialized.txt
```

Step 6: Configure kubectl (RUN only on Master Machine)

```
mkdir -p $HOME/.kube  
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config  
sudo chown $(id -u):$(id -g) $HOME/.kube/config
```

Step 7: Install Calico Network Plugin (RUN only on Master Machine)

```
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.30.3/manifests/calico.yaml
```

Step 8: Join Worker Nodes

On master:

```
cat cluster_initialized.txt
```

Copy the **kubeadm join command** and append:

```
--cri-socket unix:///var/run/cri-dockerd.sock
```

Run this command on worker nodes.

Verification

Run on master:

```
kubectl get nodes
```

Expected output:

- All nodes should be in **Ready** state

Lab Completed

You have successfully set up a Kubernetes cluster using kubeadm!

Now you can:

- Deploy applications

- Scale workloads
 - Explore Kubernetes objects
-